

Adaptive Keyframes and Evidence-Aware Hybrid Video Retrieval for AI Challenge 2026

Huy Hoang Pham Le¹, Nhat Bao Vo¹, Nguyen Phat Huynh¹, Dinh Truong Vo¹

¹Faculty of Information Technology, University of Science,
Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam.

Abstract—This paper presents AIC26-Retrieval, a hybrid video retrieval system for AI Challenge 2026 (AIC 2026). The system constructs its searchable corpus directly from raw broadcast videos using an adaptive keyframe extractor that combines shot segmentation, visual/audio saliency, stability-aware DBSCAN representatives, and temporally decayed de-duplication. A frame-time identity registry preserves the video id, original frame index, timestamp, and extracted-frame ordinal across all downstream indices. At query time, deterministic cue routing activates PE visual retrieval, OCR, ASR transcript search, and non-speech audio retrieval; weighted reciprocal rank fusion combines their ranked lists before evidence-aware video grouping. For temporal action retrieval (TRAKE), an exact dynamic-programming solution over the retrieved candidate set assembles ordered event frames instead of making greedy local selections. The deployed indices contain 382,299 keyframes, 48,819 speech segments, 466,996 audio windows, 382,299 PE vectors, and 466,996 GLAP vectors. On all 14 supplied TKIS queries, hybrid retrieval improves video-level R@1 from 50.00% to 78.57% for full queries and from 21.43% to 64.29% when only the first two sentences are available. On all four supplied TRAKE queries, the first-ranked sequence is operator-verified as correct, covering all 14 described events. The paper also reports backend runtime diagnostics, with 2.55 s median and 3.36 s p95 latency for the full hybrid path, and an implemented evidence inspection interface; interaction-time and submission-error effects are left for a controlled user study.

Index Terms—video retrieval, hybrid search, known-item search, adaptive keyframe extraction, reciprocal rank fusion, OCR, ASR, audio retrieval, temporal action retrieval

I. INTRODUCTION

Interactive video retrieval benchmarks such as the Video Browser Showdown [1], [2] and AI Challenge 2026 (AIC 2026) combine automatic ranking with human verification and final submission. This setting motivates evidence inspection, temporal navigation, and submission validation in addition to retrieval accuracy. The evaluation in this paper, however, is intentionally retrieval-centered: it measures video-level Recall@K and backend latency, while controlled interaction metrics such as time-to-correct-submit and wrong-submit rate remain future work.

AIC26-Retrieval is a retrieval console built for the AIC 2026 corpus: a heterogeneous collection of Vietnamese broadcast videos with news footage, educational programs, cooking shows, cultural programs, sports, and short-form human-interest segments. The input information need may reference visible objects, text on screen, spoken Vietnamese, non-speech sounds, ordered temporal events, or a question whose answer is present in the media. A single model rarely handles all

of these cues robustly. Dense visual encoders are effective for broad visual semantics but weak on exact text; OCR is strong for lower-thirds, tickers, and signage but blind to visual composition; ASR captures spoken content but misses music, bells, engines, and other environmental cues; audio-event models capture such cues but may be noisy when Vietnamese speech is misclassified as synthetic audio. The central design problem is to combine these channels without hiding their evidence from the operator.

AIC26-Retrieval follows an evidence-aware decomposition rather than relying on one universal embedding. It combines contrastive vision-language retrieval in the spirit of CLIP [3], HunyuanOCR [4], PhoWhisper [5], WhisperX [6], GLAP [7], and CED tags [8]. Each channel retains its attribution and supporting OCR/ASR/audio evidence. Weighted rank fusion handles incomparable score scales, video grouping prevents repeated frames from dominating the result list, and exact temporal assembly handles ordered multi-event queries.

The main contributions are:

- an adaptive keyframe construction pipeline combining shot and saliency sampling, stability-aware DBSCAN representatives, temporally decayed coverage, and a canonical frame-time registry shared by every modality;
- cue-routed visual, OCR, speech, and audio retrieval with weighted RRF and evidence-aware video grouping, evaluated on all 14 supplied TKIS queries;
- a two-pass TRAKE method that combines per-event retrieval, targeted in-video filling, and exact dynamic programming for globally ordered event assembly.

II. RELATED WORK

A. Interactive Video Retrieval

Interactive video retrieval systems combine ranked search with rapid inspection and final answer selection. The Video Browser Showdown (VBS) evaluates this setting through live known-item search tasks, where systems must support both retrieval quality and human verification [1], [2]. AIC26-Retrieval follows this operator-facing setting with grouped video results, a seekable player, and evidence display. Unlike a browser-only interface claim, our quantitative evaluation is limited to retrieval and backend diagnostics; human search time and submission accuracy remain outside the reported metrics.

B. Multimodal and Evidence-Based Search

Contrastive vision-language encoders such as CLIP enable broad text-to-image retrieval [3], but exact text, speech, and non-speech sounds are often better represented by dedicated indices. We therefore use independent visual, OCR, ASR, and audio channels, retaining the evidence that produced each hit. Their incomparable score scales are combined with reciprocal rank fusion (RRF) [9], rather than with an end-to-end learned fusion model. This design favors inspectability and modular deployment over a claim that one joint embedding solves every query type.

C. Keyframe Selection and Temporal Retrieval

Video retrieval commonly reduces a video to representative frames before indexing. Shot-transition detection, such as TransNet V2 [10], provides useful boundaries but does not by itself choose stable, diverse frames. Our offline pipeline augments such boundaries with saliency, de-duplication, and temporal coverage, then preserves a frame-time registry for every modality. For multi-event queries, standard independent top- k retrieval does not enforce chronological consistency. AIC26-Retrieval treats retrieved event candidates as a time-ordered DAG and solves the sequence-selection problem exactly over that candidate set; the TRAKE evaluation in Section X-C reports the supplied queries and isolates the resulting advantage over greedy selection.

III. PROBLEM DEFINITION AND DESIGN GOALS

A. Task Types

AIC26-Retrieval is designed around four task families, although the current implementation provides different levels of support for each.

Textual known-item search (T-KIS). The operator receives a textual description of a target moment. The answer is a video frame, submitted as `video_id + frame_idx`. Queries may mention visual content, on-screen text, speech, or sound. The system must locate plausible frames, expose evidence, and allow fast video verification.

Visual known-item search (V-KIS). The operator searches for a visual target. The current implementation provides partial V-KIS support through the same dense visual retrieval and inspection pipeline as KIS; true image-query or clip-query V-KIS input is left as an extension point.

Question answering (QA). The operator must locate a relevant moment and provide a short textual answer. The current system treats QA as locate-plus-text evidence: OCR, speech, and audio snippets are surfaced, and the answer is edited manually by the operator. We intentionally do not claim image-based VLM chain-of-thought answering in the live system.

Temporal action retrieval (TRAKE). A query describes a sequence of events that must occur in one video in chronological order. The output is an ordered list of event frames. TRAKE differs from standard top- k retrieval: a frame that is locally strong for event E_i may be unusable if it prevents a valid increasing sequence with E_{i+1} .

B. Design Goals

The following goals shaped the architecture.

G1: Stable identity over local paths. Every candidate must be represented by a canonical public identifier:

```
<category>/<video_id>/<frame_3digits>.
```

This identifier is also the image identifier. Local extraction paths from Kaggle or other machines are never used for routing, cache keys, media lookup, or result handling. Final result records use `video_id` and `frame_idx`.

G2: Channel specialization with graceful degradation. Each modality is served by an adapter with a mock fallback. If PE, GLAP, Elastic, or an LLM is unavailable, the system reports the degraded capability but preserves any working channels rather than failing the entire search.

G3: Evidence-first ranking. Fusion should combine complementary signals while preserving channel attribution and snippets. The operator must be able to understand why a candidate appears in the ranking.

G4: Video-level inspection. In a live search setting, candidates are verified in video context. The interface groups frames by video, opens an inline video at the selected timestamp, and provides a lightweight keyframe timeline.

G5: Submission safety. Since wrong or duplicate submissions are costly, the system verifies `video_id`, `frame_idx`, timestamp, task type, QA answer, event order, and duplicate keys before submission.

G6: Temporal optimality for sequences. TRAKE assembly should maximize event coverage and relevance under strict temporal order. A dynamic program is used instead of a greedy sequence builder.

IV. SYSTEM OVERVIEW

Fig. 1 summarizes the architecture. Offline jobs extract and normalize multimodal metadata, upload sparse metadata to Elastic Cloud, dense vectors to Milvus/Zilliz, and media to Cloudflare R2. The online FastAPI backend parses queries, runs enabled channels in parallel, fuses results, enriches frame timing from the keyframe map, and returns evidence-rich video groups. The React/Vite frontend presents a single-screen search console optimized for a human operator.

A. Offline Indexing

Offline indexing produces four Elastic indices and two Milvus collections. Elastic stores keyframe timing, OCR records, ASR segments, and audio-window metadata [11]. Milvus stores dense PE visual embeddings and GLAP audio embeddings [12]. Cloudflare R2 stores public keyframe and video media. The indexing stage is repeatable: raw extraction outputs are preserved, staging JSONL files are regenerated by `keyframe_mapping.py`, and `elastic_upload.py` and `milvus_upload.py` support retry/resume semantics.

B. Online Retrieval

The backend executes the online path:

- 1) parse the query into a routing object;

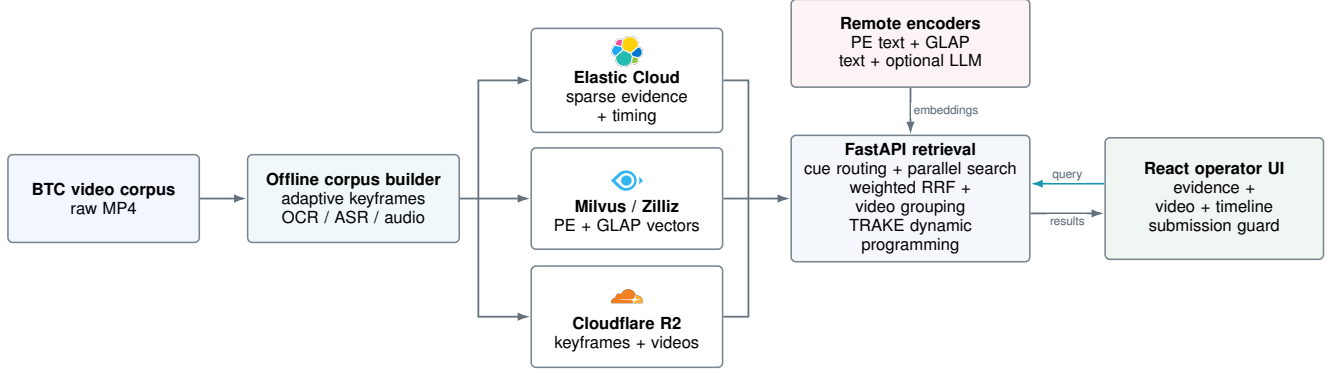


Fig. 1. End-to-end architecture of AIC26-Retrieval. Offline components build indices and media URLs; the online backend performs query routing, parallel retrieval, fusion, video grouping, and TRAKE assembly, while the operator interface exposes evidence, video context, and guarded submission.

- 2) run enabled channels in parallel;
- 3) fuse ranked channel lists by RRF;
- 4) batch-enrich timing metadata from the keyframe map;
- 5) group frames by video and serialize media URLs and evidence;
- 6) for TRAKE, assemble ordered event sequences.

The frontend consumes only backend APIs; secrets remain in the backend process.

C. Interface and Result Inspection

The interface is a three-column console: query controls and channel routing on the left, grouped results with video and timeline in the center, and detailed evidence, TRAKE slots, submit guard, and history on the right. Shortcuts support the critical loop: focus query, search, move between videos and frames, open the video with `v`, pause to pick a raw frame for TRAKE, inspect evidence, and submit.

V. OFFLINE MULTIMODAL INDEXING

Fig. 2 shows the offline dataflow. The deployed search path starts from BTC videos, extracts its own representative keyframes with the adaptive algorithm described below, and indexes those extracted frames for retrieval and submission. The keyframe map produced by this stage stores the original video frame index, presentation timestamp, FPS, and stable frame ordinal, so every downstream OCR, visual, caption, object, speech, or audio hit can be mapped back to the same `video_id + frame_idx` submission contract.

A. Keyframe Identity and Temporal Mapping

The identity layer is a core contribution because many extraction artifacts use environment-dependent paths. For example, OCR records may contain absolute Kaggle paths, while PE embeddings are keyed by dataset-relative paths. These paths are useful for provenance but unsafe for routing. AIC26-Retrieval normalizes every public frame as:

$$\text{submit_keyframe_id} = C/V/N,$$

TABLE I
CORE IDENTITY AND JOIN FIELDS.

Field	Meaning	Used by
<code>submit_keyframe_id</code>	Public frame id, category/video/frame; image_id	all result frames, keyframe map, OCR, Milvus image grouping, media URL, timeline, submit
<code>video_id</code>	Video stem, e.g., K01_V001	media URL, timeline
<code>keyframe_n</code>	Extracted keyframe ordinal, e.g., 1 for 001.jpg	media URL, timeline
<code>frame_idx</code>	Absolute video frame index	KIS/QA/V-KIS/TRAKE result
<code>pts_time</code>	Presentation timestamp in seconds	video seek, timeline, TRAKE order
<code>window_id</code>	Audio-window primary key	Elastic audio ↔ Milvus GLAP join
<code>segment_id</code>	Speech segment primary key	Elastic speech

where C is the submit category, V is the video id, and N is the three-digit ordinal assigned by the extraction pipeline. The L26 shard variants are normalized to category L26 for display and submission. Internally, joins use `submit_keyframe_id` or the pair `(video_id, keyframe_n)`. The keyframe map stores `pts_time`, `FPS`, and `frame_idx` for every extracted keyframe and is used to map OCR frames, speech segments, audio windows, video timeline entries, and submissions.

The media URL contract mirrors the same identity:

keyframe: `B/Keyframes/Keyframes_C/V/N.jpg`,
video: `B/Videos/Videos_C/V.mp4`,

where B is `MEDIA_BASE_URL` and $C = \text{group_from_video_id}(V)$.

Table I summarizes the canonical fields used to connect all modalities to retrieval, media display, and submission.

B. Adaptive Keyframe Extraction

Adaptive keyframe extraction is the first offline stage of AIC26-Retrieval, not merely an auxiliary analysis branch. The searchable image set, OCR frames, PE visual embeddings, visual captions, and object detections are built on the representative frames produced by this extractor. The extractor

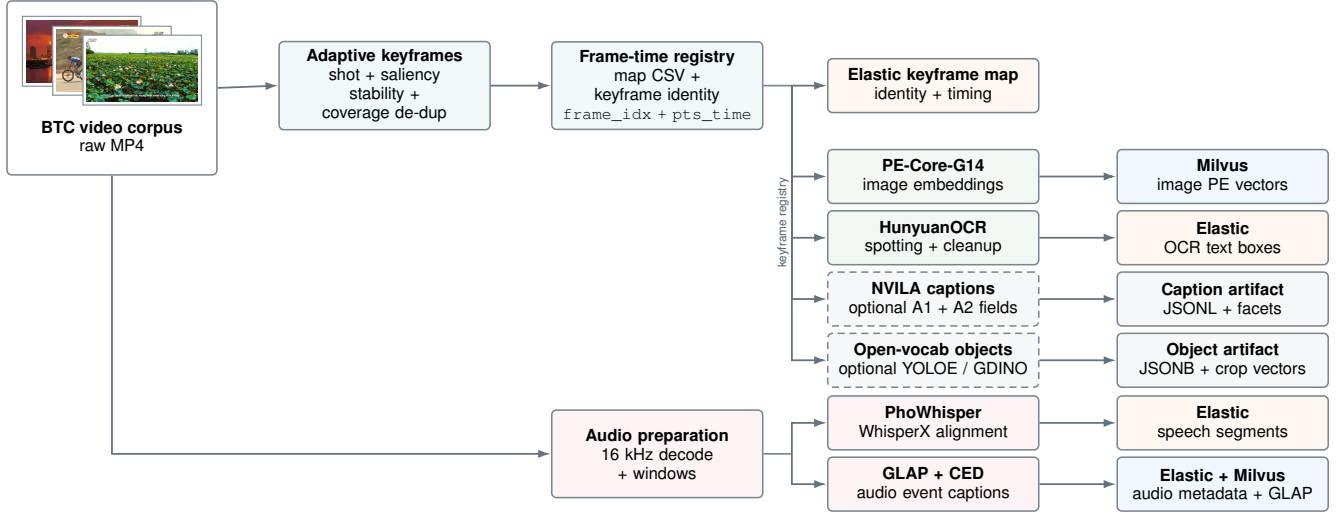


Fig. 2. Offline indexing and enrichment pipeline. The system extracts its own representative keyframes from BTC videos, maps every extracted frame to `video_id + frame_idx`, uploads sparse text/temporal metadata to Elastic, and uploads dense image/audio vectors to Milvus. Caption and object branches share the same identity registry and are optional offline enrichment extensions.

enforces the same evidence contract as the online system: every extracted frame preserves `video_id`, `frame_idx`, `pts_time`, FPS, and keyframe ordinal so it can be displayed, searched, and submitted without relying on environment-specific paths.

The extractor first partitions a video into shots with a three-level fallback stack. TransNetV2 [10] is the preferred detector when its weights are available. If it fails, PySceneDetect-style content differences are used; if that also fails, the pipeline computes HSV histogram change at a low sampling rate and marks large Bhattacharyya-distance jumps as cuts. Shots shorter than 0.6 seconds are merged into a neighbor, including the first shot, to avoid pathological fragments and transition frames.

Within each shot, sampling is driven by an event-saliency signal

$$S(t) = \text{ReLU}(\alpha \Delta_v(t) + \beta \Delta_a(t) - \epsilon), \quad (1)$$

where $\Delta_v(t)$ is a low-resolution HSV visual-change score and $\Delta_a(t)$ is the magnitude of the derivative of 100 ms audio RMS energy. Both signals are normalized per shot by a high percentile so a quiet interview and a visually dynamic outdoor report are compared against their own local background. If audio extraction fails or the video has no usable audio stream, the pipeline sets $\beta = 0$ and continues with visual saliency only.

The candidate set is the union of a background grid and event-focused samples. For short shots, the grid keeps one to five fractional positions; for shots longer than 20 seconds, it samples at a configurable candidate FPS. Event samples are added around local maxima of $S(t)$ within a 0.25 second window. A small boundary margin removes frames close to shot transitions, and a per-shot cap of 40 candidates keeps GPU encoding predictable.

Candidate frames are decoded by linear traversal rather than random seeking: the video reader calls `grab()` for most frames and `retrieve()` only for selected indices. This matters for compressed MP4 files because random seeks may force repeated decoding from nearby I-frames. Each candidate receives a stability score

$$q(f) = 0.50q_{\text{sharp}}(f) + 0.30q_{\text{bright}}(f) + 0.20q_{\text{contrast}}(f), \quad (2)$$

where sharpness is derived from Laplacian variance, brightness uses a Gaussian preference around mid-level exposure, and contrast uses grayscale standard deviation. The score is used for tie-breaking, not as a hard deletion rule.

Visual features are encoded with PE-Core-G14-448 [13] when available, with SigLIP [14] or handcrafted HSV/edge features as fallbacks. The system runs one model copy per GPU in Kaggle T4x2 sessions and L2-normalizes all vectors. Per-shot DBSCAN [15] over cosine distance selects representative clusters without predefining the number of keyframes. For a cluster, the selected frame maximizes

$$0.70\langle e_i, \bar{e}_C \rangle + 0.30q(f_i), \quad (3)$$

combining representativeness and stability. Stable DBSCAN noise points are kept only when they are sufficiently dissimilar to representatives already selected for the shot.

The final video-level de-duplication uses temporally decayed coverage:

$$\text{cov}(f, K) = \max_{k \in K} \cos(e_f, e_k) \exp\left(-\frac{|t_f - t_k|}{\tau}\right). \quad (4)$$

Frame f is kept when $\text{cov}(f, K) < 0.85$ with $\tau = 60$ seconds. This rule suppresses near duplicates inside the same event while preserving visually similar scenes that recur minutes apart. The extractor writes `keyframes/`,

Algorithm 1 Adaptive representative keyframe extraction

Require: Video V , configuration Θ
Ensure: Keyframes, map CSV, and feature rows

- 1: $shots \leftarrow \text{TransNetV2}(V)$ if available
- 2: **if** $shots$ invalid **then**
- 3: $shots \leftarrow \text{content-based shot detection}(V)$
- 4: **end if**
- 5: **if** $shots$ invalid **then**
- 6: $shots \leftarrow \text{HSV histogram cuts}(V)$
- 7: **end if**
- 8: Merge shots shorter than $\Theta.minShotSec$
- 9: **for all** shot s in $shots$ **do**
- 10: Compute low-resolution visual deltas $\Delta_v(t)$
- 11: Compute audio RMS derivative $\Delta_a(t)$ if audio exists
- 12: Normalize Δ_v, Δ_a per shot
- 13: $S(t) \leftarrow \text{ReLU}(\alpha\Delta_v(t) + \beta\Delta_a(t) - \epsilon)$
- 14: $C \leftarrow$ background grid samples for s
- 15: $C \leftarrow C \cup$ samples near local maxima of $S(t)$
- 16: Remove candidates inside transition boundary margin
- 17: Limit C to $\Theta.maxCandidatesPerShot$
- 18: Decode candidate frames by linear grab/retrieve
- 19: Compute stability $q(f)$ for every $f \in C$
- 20: Encode candidate embeddings e_f and L2-normalize
- 21: Cluster embeddings with per-shot DBSCAN
- 22: Select one stable representative per cluster
- 23: Keep stable DBSCAN-noise frames only if visually distinct
- 24: **end for**
- 25: Sort representatives by stability and apply temporally decayed coverage
- 26: Write $keyframes/, map-keyframes/, embeddings/$

$map-keyframes/$, and $embeddings/$ artifacts with $video_id, frame_idx, pts_time, shot\ id$, and feature-row provenance.

C. Structured Visual Captioning

Structured visual captioning is an optional offline enrichment extension over the extracted keyframe registry; the core image, OCR, speech, and audio retrieval paths do not depend on it. When enabled, it adds filterable scene facets and searchable descriptions without changing frame identity. The notebook implementation uses NVILA-Lite-2B [16], which fits on a single T4 and can shard the corpus by video-prefix range. It writes a resumable `captions.jsonl` artifact and skips completed `keyframe_id` values on reruns.

The captioner runs two passes. Pass A1 labels one keyframe with a strict JSON schema. **Hard facets** are `scene_type`, `location_type`, `time_of_day`, `weather`, `camera_shot`, and `num_people`. Their values come from small controlled vocabularies so they can be used as filters. **Soft descriptions** are `event_type`, `people_roles`, `people_description`, `key_objects`, `dominant_colors`, `distinctive_detail`, `summary`, and `text_on_screen`. Parser code extracts the first valid JSON object, coerces out-of-vocabulary hard values to `unknown` (or 0 for people count), and records uncertain values for later auditing.

Pass A2 optionally feeds a short neighborhood around the keyframe to identify motion or action tags such as speaking, walking, vehicle moving, fire spreading, or static. This pass captures evidence that a single frame cannot express. Near duplicates are grouped by difference hash, so repeated frames can inherit the representative’s metadata rather than spending VLM budget repeatedly.

Algorithm 2 Structured visual captioning extension

Require: Extracted keyframe records $\mathcal{K}$
Ensure: Caption JSONL and optional caption/image vectors

- 1: Load completed `keyframe_ids` from existing JSONL
- 2: **for all** video shard V **do**
- 3: Read ordered keyframe refs from `map-keyframes`
- 4: Group near duplicates by difference hash
- 5: **for all** keyframe k not yet completed **do**
- 6: Run A1 single-image prompt with strict JSON schema
- 7: Extract JSON; coerce hard fields to allowed vocabularies
- 8: **if** A2 enabled **then**
- 9: Run A2 neighborhood prompt for `action_motion`
- 10: **end if**
- 11: Merge identity, A1/A2 fields, dedup group, and model version
- 12: Append one JSONL record and flush
- 13: **end for**
- 14: **end for**
- 15: Build facets and optional semantic embedding indices

The optional caption index is split by retrieval use. For V-KIS, the prompt clip is observed by an operator and on-screen text may be removed, so `text_on_screen` is excluded from V-KIS soft matching and retained for T-KIS/QA. The text-independent fields are concatenated into a caption string and embedded with a multilingual sentence encoder for text-to-caption search. A separate optional SigLIP image-vector index supports text-to-image search from operator descriptions. The facet table supports hard filtering before ranking, while caption vectors support soft semantic retrieval.

D. Open-vocabulary Object Detection

Closed-set detectors trained on COCO-like labels miss broadcast-specific entities: Vietnamese TV logos, lower-third graphics, local signs, uniforms, anchors, emergency vehicles, ceremonial objects, and rare news objects. The project therefore treats object detection as an optional zero-shot, no-fine-tuning enrichment branch over keyframes, not as a dependency of the core hybrid retriever. Its evidence record contains `label`, `bbox`, `conf`, `source`, `color`, `grid_cell`, optional `crop_emb_id`, and per-label counts.

The full design uses an ensemble. YOLOE [17] provides fast open-vocabulary bulk tagging; Grounding DINO [18] or MM-Grounding-DINO provides prompt-specific grounding for rarer concepts; a Qwen-style VLM grounding pass is reserved for dense or ambiguous cases such as counting crowds. The Kaggle notebook defaults to YOLOE-only for throughput and exposes an A/B cell that estimates the additional boxes and labels contributed by the ensemble before deciding whether the extra runtime is worth it.

Prompt vocabulary is genre-conditional. A base vocabulary covers common objects such as person, crowd, text on screen, logo, microphone, sign, table, car, and motorbike. Genre-specific vocabularies are selected from the video prefix: news, cycling, lion dance, exam, cooking, culture, Mekong, and positive shorts use different concept lists. This reduces detector noise and avoids wasting prompt budget on objects that are implausible for a program genre.

When multiple detectors are enabled, overlapping boxes are fused with weighted box fusion [19]. For boxes with the same

Algorithm 3 Zero-shot object enrichment

Require: Extracted keyframes $\mathcal{K}$, genre vocabularies

Ensure: Detection JSONL and optional crop embeddings

```
1: for all keyframe  $k$  do
2:   Infer genre and build the allowed concept list
3:   Run YOLOE open-vocabulary detection
4:   if ensemble enabled then
5:     Run Grounding-DINO on concept chunks
6:     Fuse same-label overlaps with WBF
7:   end if
8:   for all detection  $d$  do
9:     Store label, confidence, source list, and bbox
10:    Derive 7x7 grid cell and dominant crop color
11:    if  $\text{conf}(d) \geq \Theta.\text{cropConf}$  then
12:      Encode crop and store crop_emb_id
13:    end if
14:  end for
15:  Store label counts and append the JSONL record
16: end for
17: Validate with weak references and rendered bbox audits
```

label and intersection-over-union above a threshold, the fused box is

$$b^* = \frac{\sum_i s_i w_i b_i}{\sum_i s_i w_i}, \quad (5)$$

where s_i is detector confidence and w_i is a model weight. This preserves agreement across detectors instead of discarding boxes as non-maximum suppression would. Boxes seen by only one detector may be retained with lower confidence for recall.

Each detection is enriched with attributes that are cheap but useful for compositional queries. The bbox center is mapped to a 7x7 spatial grid for phrases such as “upper left” or “right side”. Dominant color is computed by cropping the box and quantizing to a 16-color palette. Counts are stored per label, with dense scenes optionally delegated to a VLM counter. High-confidence crops are encoded with DINOv2-style crop embeddings [20] for instance search, such as finding the same logo or vehicle shape again. Metadata can be stored as JSONB-style records, while crop embeddings enter a separate Milvus collection.

Because the project has no exhaustive object labels, validation is weakly supervised. The branch compares overlapping labels against BTC-provided object metadata where available, runs manual spot checks on random keyframes, monitors cross-model consistency, and renders bbox overlays for qualitative vocabulary and threshold tuning. The objective is not to claim mAP, but to choose usable confidence thresholds and genre vocabularies.

E. Visual Embedding Index

The visual channel uses PE-Core-G14-448 embeddings [13], following the general contrastive text-image retrieval paradigm popularized by CLIP [3]. The image collection `aic26_image_peg14_v1` contains 382,299 vectors of dimension 1280 and uses cosine similarity. Each Milvus entity stores the canonical id, category, video id, keyframe number, provenance path, and vector. At query time, a PE text encoder endpoint converts the English visual query into a vector, and Milvus returns ranked keyframes. When query expansion is enabled, several compact English visual variants are encoded

and searched independently; results are merged by maximum cosine score per frame before entering the channel-fusion stage.

F. OCR Index

OCR is extracted with HunyuanOCR [4] in text-spotting mode. The pipeline parses model outputs into text boxes, normalizes Unicode to NFC, creates a folded diacritic-free representation, and writes one record per keyframe. A post-processing step removes boxes that are pure broadcast clock, station logo, watermark, YouTube button, single-character noise, or coordinate remnants, while preserving text that appears inside meaningful sentences. It adds `text_clean`, `text_clean_fold`, `clock`, and `hour`. The Elastic OCR index contains 382,299 records. The retrieval adapter searches `text_clean`, `text_nfc`, and fuzzy folded text, with optional hour and clock filters. Empty OCR queries never fall through to match-all.

G. Speech Transcript Index

Speech extraction begins by decoding each MP4 to 16 kHz mono audio. PhoWhisper-large [5] performs Vietnamese ASR through CTranslate2, and WhisperX forced alignment assigns word-level timestamps and alignment scores [6]. A faster-Whisper path remains as a fallback when the aligner is unavailable. Extraction is checkpointed per video: each `.speech.json` file stores the transcript text, segment start/end, word timestamps, word scores, and backend-specific confidence metadata.

Index construction, detailed in Algorithm 4, deliberately keeps uncertain segments so thresholds can be changed without rerunning ASR. The mapper creates a stable `segment_id`, computes `avg_word_score`, and assigns confidence as low below 0.40, mid in $[0.40, 0.50)$, high at or above 0.50, or missing. It also labels each segment as intro, preview, or body. The segment start, center, and end times are mapped independently to the nearest extracted keyframes; the center mapping becomes the canonical `submit_keyframe_id` while all three anchors remain available for temporal inspection and TRAKE.

The Elastic index contains 48,819 segments across 1,478 videos. Vietnamese text queries use full-text and boosted phrase matching. Before channel fusion, the Elastic score is multiplied by confidence factors $\{1.0, 0.75, 0.25, 0.5\}$ for high, mid, low, and missing confidence, and role factors $\{1.0, 0.7, 0.6\}$ for body, intro, and preview. Thus weak or promotional segments remain retrievable but are less likely to become incorrect temporal anchors. Fig. 3 illustrates the complete mapping from MP4 audio to the searchable speech record.

H. Audio Event Index

The non-speech audio branch targets cues that ASR cannot represent, such as music, applause, bells, engines, animal sounds, and kitchen events. After the same 16 kHz conversion, each video is divided into five-second windows with a

Algorithm 4 Speech transcript indexing

Require: Video V , extracted-keyframe registry K
Ensure: Mapped speech-segment records for Elastic

- 1: $x \leftarrow$ decode V as 16 kHz mono audio
- 2: $S \leftarrow$ PhoWhisper-ASR(x)
- 3: $S \leftarrow$ WhisperX-Align(x, S) if available
- 4: **for all** segment $s \in S$ **do**
- 5: Normalize text and retain word timestamps/scores
- 6: Assign confidence bucket and temporal role
- 7: $k_s \leftarrow$ NEARESTKEYFRAME($K, V, s.start$)
- 8: $k_c \leftarrow$ NEARESTKEYFRAME($K, V, s.center$)
- 9: $k_e \leftarrow$ NEARESTKEYFRAME($K, V, s.end$)
- 10: Build stable `segment_id` from video and span
- 11: Set canonical submit id from k_c ; retain k_s, k_c, k_e
- 12: Append segment without hard-deleting low confidence
- 13: **end for**
- 14: Bulk-upload records to the Elastic speech index

2.5-second hop. Every window receives a normalized 1024-dimensional GLAP embedding [7] and the top five CED tags [8] with scores. A MiDashengLM caption [21] is generated only when the highest-scoring tag passes the non-speech gate, limiting caption cost and preventing narration from dominating the caption branch. The extractor writes one `.audio.json` record per window and one row in a float16 `.glap.npy` matrix, connected by `glap_idx`.

Algorithm 5 describes index construction. The mapper creates a stable `window_id`, maps window start, center, and end to the frame-time registry, and derives quality fields before upload. Captions are classified as useful, generic, Vietnamese-ASR leakage, or absent. CED labels known to be noisy on Vietnamese narration, especially *Speech synthesizer* and *Mantra*, set stoplist flags rather than modifying the raw extraction artifact. This separation makes the extraction reproducible while allowing the retrieval policy to evolve.

The resulting corpus contains 466,996 windows and 35,222 caption records. Elastic stores timing, tags, scores, captions, quality flags, and keyframe anchors; Milvus stores GLAP vectors keyed by the same `window_id`. Online retrieval therefore has two complementary ranked lists: Elastic matches exact labels and caption text, while GLAP matches a semantic sound query in vector space. A stoplisted top-1 label is dropped, weaker quality signals are demoted, and the surviving Elastic and GLAP lists are merged by rank-RRF with $k = 30$ before entering global multi-channel fusion.

VI. ONLINE QUERY PROCESSING AND RETRIEVAL

A. Query Understanding

The parser produces a structured routing object with task type, channel enable flags, channel weights, channel-specific query strings, filters, TRAKE events, QA settings, and UI hints. There are two parser paths.

Heuristic parser. The deterministic parser detects OCR-like queries from digits, visible-text cues, quotes, or clock patterns; speech-like queries from verbs such as *peak*, *announce*, or *interview*; audio queries from non-speech sound words; and TRAKE queries from explicit event markers or chronological connectors. It enables image search by default,

Algorithm 5 Audio-event index construction

Require: Video V , extracted-keyframe registry K
Ensure: Elastic audio records and Milvus GLAP vectors

- 1: $x \leftarrow$ decode V as 16 kHz mono audio
- 2: $W \leftarrow$ five-second windows with 2.5-second hop
- 3: **for all** window $w_i \in W$ **do**
- 4: $g_i \leftarrow$ L2-normalized GLAP embedding of w_i
- 5: $T_i \leftarrow$ CED top-5 labels and scores for w_i
- 6: **if** top-1 label is non-speech and passes caption gate **then**
- 7: $p_i \leftarrow$ MiDashengLM caption(w_i)
- 8: **else**
- 9: $p_i \leftarrow \emptyset$
- 10: **end if**
- 11: Classify caption quality and mark stoplist hits
- 12: Map start, center, and end to nearest keyframes in K
- 13: Build `window_id`; append metadata with `glap_idx=i`
- 14: Store g_i as row i of the video GLAP shard
- 15: **end for**
- 16: Upload metadata to Elastic and vectors to Milvus by window id

Algorithm 6 Heuristic query routing

Require: Query q , task hint h , previous hints P

- 1: $x \leftarrow$ concatenate(P, q)
- 2: Enable image channel with visual query x
- 3: **if** x contains digits, quotes, clock, or visible-text words **then**
- 4: Enable OCR with exact, folded, and time filters
- 5: **end if**
- 6: **if** x contains speech verbs **then**
- 7: Enable speech with Vietnamese query x
- 8: **end if**
- 9: **if** x contains non-speech sound words **then**
- 10: Enable audio with English sound labels if known
- 11: **end if**
- 12: **if** h is TRAKE or x has event connectors **then**
- 13: Split q into ordered events $E_1, \dots, E_n$
- 14: Create per-event visual/OCR/speech/audio fields
- 15: **end if**
- 16: Apply manual channel overrides
- 17: **if** all channels disabled **then**
- 18: Re-enable image channel
- 19: **end if**
- 20: Return routing JSON

adds OCR/speech/audio only when cues are present, and guarantees that at least one retrieval channel remains active.

LLM parser. When enabled, the backend calls an OpenAI-compatible NVIDIA NIM endpoint. The LLM returns JSON under a strict schema and may generate compact English visual paraphrases for dense image search. If the LLM is unavailable or produces invalid JSON after retry, the system falls back to heuristics. When the LLM is disabled, Vietnamese queries are translated to English for the PE visual channel; OCR and speech remain in Vietnamese. This translation is a best-effort, external machine-translation service in the implementation; if it fails or the input already appears to be English, the original string is used unchanged. No online translation is used for the reported TKIS evaluation.

B. Multi-channel Retrieval

Fig. 4 details online retrieval. Each enabled channel returns ranked `ChannelHit` objects containing `submit_keyframe_id`, `video_id`, `keyframe_n`, `rank`, `raw score`, `timestamp` if available, and `evidence`.

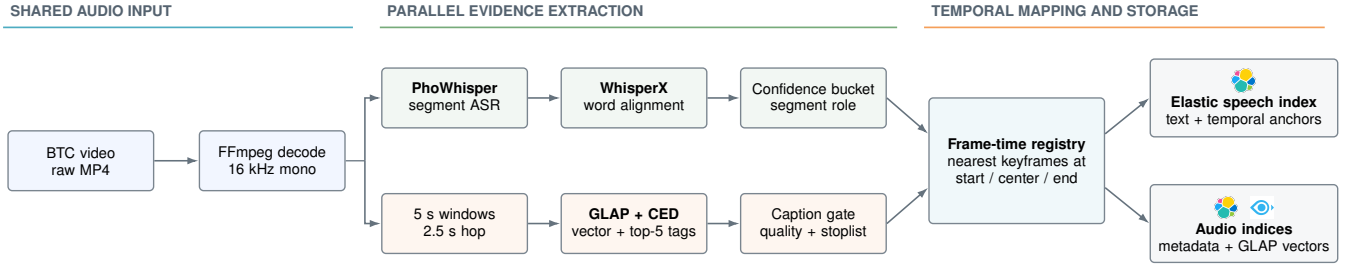


Fig. 3. Speech and audio-event index construction. Both branches share the decoded waveform and frame-time registry but preserve different evidence: aligned Vietnamese transcript segments in Elastic, and audio tags/captions in Elastic plus GLAP vectors in Milvus. Mapping each time span at its start, center, and end keeps retrieval evidence connected to inspectable and submit-safe keyframes.

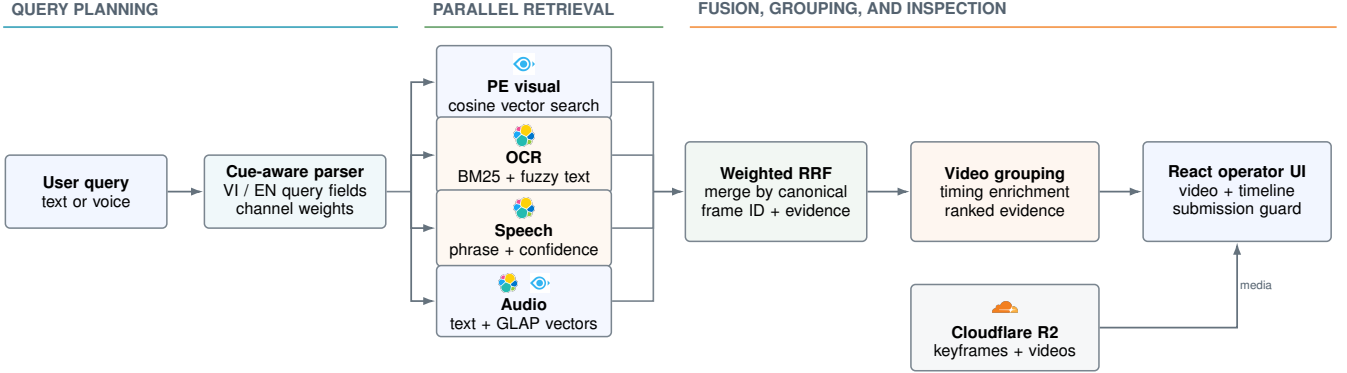


Fig. 4. Online query execution. The parser creates a channel plan; channels run in parallel; ranked lists are fused by RRF and grouped for video-centric inspection, while Cloudflare R2 supplies keyframe and video media to the UI.

Image PE. The channel encodes English visual queries, searches Milvus, and converts hits into frame candidates. With expansion, each paraphrase is searched separately and only the best cosine per frame is kept.

OCR. The channel searches clean Vietnamese text, NFC text, and folded fuzzy text, with optional exact phrases and clock/hour filters.

Speech. The channel searches ASR segments and demotes lower-confidence or structurally less useful segments.

Audio. Elastic audio search ranks tag and caption matches after stoplist/caption-quality filtering. GLAP vector hits are searched in Milvus when the audio text encoder is available. The two audio lists are fused by a small- k rank fusion into a single audio channel list.

C. Reciprocal Rank Fusion

Given channel ranked lists $\mathcal{L}_c$, each frame d receives the reciprocal rank fusion score [9]

$$s_{\text{RRF}}(d) = \sum_c \mathbb{I}[d \in \mathcal{L}_c] \frac{w_c}{k + r_c(d) + 1},$$

where $r_c(d)$ is the zero-based rank of d in channel c , w_c is the channel weight from the parser/operator, and $k = 60$ by default. Fusion is performed over `submit_keyframe_id`, so evidence from multiple channels attaches to the same canonical frame. Relevance feedback applies deterministic post-fusion edits: positive videos receive a small boost, negative frames are dropped, and negative videos are demoted.

Algorithm 7 Channel fusion and video grouping

Require: Ranked lists $\{\mathcal{L}_c\}$, channel weights $\{w_c\}$

- 1: Initialize map F from `submit_keyframe_id` to fused frame
- 2: **for all** channels c **do**
- 3: **for all** hit d at zero-based rank r in $\mathcal{L}_c$ **do**
- 4: $F[d].score \leftarrow F[d].score + w_c / (k + r + 1)$
- 5: Attach channel badge and evidence to $F[d]$
- 6: **end for**
- 7: **end for**
- 8: Apply frame/video feedback edits
- 9: Batch-fetch missing `pts_time`, `frame_idx`, and `FPS`
- 10: Group fused frames by `video_id`
- 11: **for all** video groups V **do**
- 12: Sort frames and compute aggregate group score
- 13: Mark ambiguous groups with a large temporal gap
- 14: **end for**
- 15: Return video groups sorted by composite score

D. Video-level Grouping and Evidence Aggregation

Fused frames are grouped by `video_id`. A video score is computed from the strongest frame, the mean of the top frames, and temporal support around the best frame:

$$S(V) = 0.75 \cdot \frac{s_{\max}(V)}{s_{\max}^{\text{global}}} + 0.15 \cdot \frac{\text{meanTop3}(V)}{s_{\max}^{\text{global}}} + 0.10 \cdot \text{clusterSupport}(V).$$

This coverage-aware but best-frame-dominant score prevents videos with many weak hits from burying a video with a single strong hit. If the top timestamps within a video are separated by a large gap, the group is marked ambiguous so

the operator knows to inspect multiple temporal clusters. Here, $\text{clusterSupport}(V)$ is the normalized local support around the best frame: the system counts fused frames whose timestamps fall within ± 30 seconds of the highest-scoring frame, subtracts the best frame itself, and divides by four, clipping the result to $[0, 1]$. Ambiguity is marked when the sorted timestamps of the top five frames contain a gap larger than 30 seconds. The grouping weights 0.75/0.15/0.10 are heuristic values selected for operator-oriented ranking and were not learned from the 14-query evaluation set.

VII. TEMPORAL ACTION RETRIEVAL FOR TRAKE

A. Event Decomposition

A TRAKE query is parsed into events $E_1, \dots, E_n$. Explicit markers such as E1 : and E2 : are preferred; otherwise chronological connectors such as “then”, “after that”, or Vietnamese equivalents are used. Each event receives visual, OCR, speech, and audio query fields. The default ordering rule is strict increasing time within one video.

B. Per-event Retrieval

The TRAKE service runs the normal retrieval stack for each event independently and concurrently, using a wider candidate pool than standard KIS. This yields per-event frame lists $\mathcal{F}_1, \dots, \mathcal{F}_n$. Candidate frames without timestamps are excluded from sequence assembly because the temporal constraint cannot be evaluated.

C. Dynamic Programming Sequence Assembly

For each video, event candidates form a directed acyclic graph ordered by event index and timestamp. An edge from candidate u to candidate v exists when $u.\text{event} < v.\text{event}$ and $u.\text{time} < v.\text{time}$. The objective is to maximize event coverage first and relevance second, represented directly as the tuple (coverage, relevance) and compared lexicographically. Thus, adding one covered event always dominates any relevance difference among chains with fewer covered events. The exact dynamic program over the retrieved candidate set is shown in Algorithm 8.

Complexity. For one video, let $M = |N|$ be the total number of retained event candidates. Sorting the candidates costs $O(M \log M)$. The implementation examines every earlier candidate for each node, so the dynamic program costs $O(M^2)$ time; the value and parent arrays require $O(M)$ memory. With n events and the deployed per-event, per-video cap $c = 12$, $M \leq nc$, giving $O(n^2 c^2)$ time and $O(nc)$ memory per video. Across a set of candidate videos $\mathcal{V}$, sequence assembly costs $O(\sum_{V \in \mathcal{V}} M_V^2)$ after sorting. Encoder and index-search latency belongs to per-event retrieval and is not included in this DP bound.

D. Two-pass In-video Fill

After pass 1, some promising videos cover only a subset of events. The service then performs targeted in-video searches for missing events using Milvus filters on `video_id`. A fill candidate must exceed a relative similarity floor and, when

Algorithm 8 TRAKE lexicographic increasing-chain DP

Require: Candidate lists $\mathcal{F}_1, \dots, \mathcal{F}_n$

Ensure: One ordered pick per covered event

```

1:  $N \leftarrow$  all candidates sorted by  $(\text{event}, \text{time})$ 
2: for all  $i \in 1, \dots, |N|$  do
3:    $dp[i] \leftarrow (1, \text{score}(N_i))$ 
4:    $\text{parent}[i] \leftarrow \perp$ 
5:   for all  $j < i$  do
6:     if  $\text{event}(N_j) < \text{event}(N_i)$  and  $\text{time}(N_j) <$ 
        $\text{time}(N_i)$  then
7:        $\text{cand} \leftarrow (dp[j].\text{coverage} +$ 
        $1, dp[j].\text{relevance} + \text{score}(N_i))$ 
8:       if  $\text{cand} >_{\text{lex}} dp[i]$  then
9:          $dp[i] \leftarrow \text{cand}$ 
10:         $\text{parent}[i] \leftarrow j$ 
11:      end if
12:    end if
13:  end for
14: end for
15: Trace back from  $\arg \max_i dp[i]$ 
16: Return selected candidates indexed by event

```

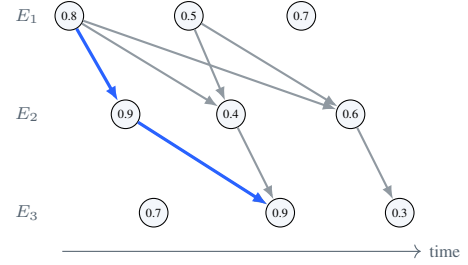


Fig. 5. TRAKE candidates form a DAG over event index and time. The selected path maximizes coverage before relevance; it may skip locally high candidates that block a better ordered chain.

possible, fit temporally between already-covered neighboring events. Filled candidates are marked `via_fill` and assigned a small fallback score. Video sequence ranking prioritizes *confident coverage*, i.e., events found by genuine pass-1 retrieval, before total coverage and relevance. Thus a video whose apparent full coverage is supported primarily by lower-confidence second-pass fill candidates cannot outrank one with more direct pass-1 evidence.

VIII. OPERATOR INTERFACE AND SUBMISSION GUARD

A. Search Console

Fig. 7 presents the interface layout. The left column contains the task selector, query box, previous hints, voice input, channel toggles, and query-understanding panel. The center column contains grouped or flat results, the inline video player, and timeline. The right column contains frame details, evidence snippets, TRAKE slots, submit guard, and submission history.

B. Evidence Inspection

Each candidate frame displays channel badges and snippets. OCR evidence includes clean text and optional clock/hour

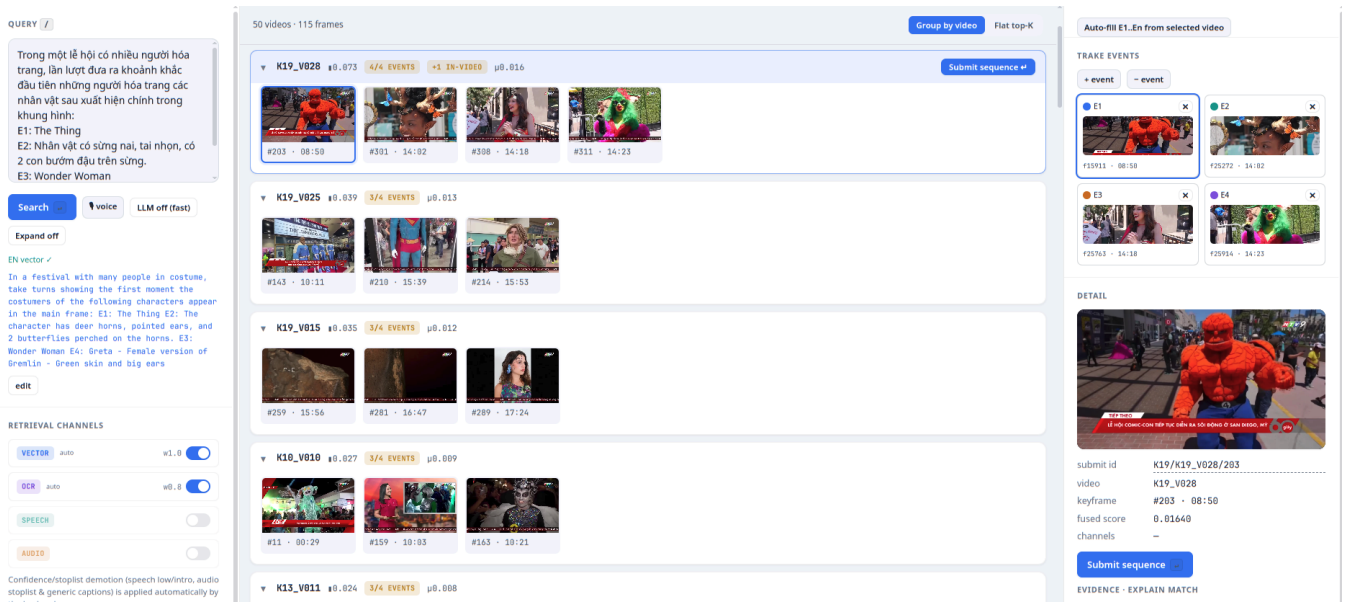


Fig. 6. TRAKE interface for inspecting ordered event candidates. The UI keeps event slots, candidate evidence, video context, and submit controls visible so the operator can verify temporal order before submitting a sequence.

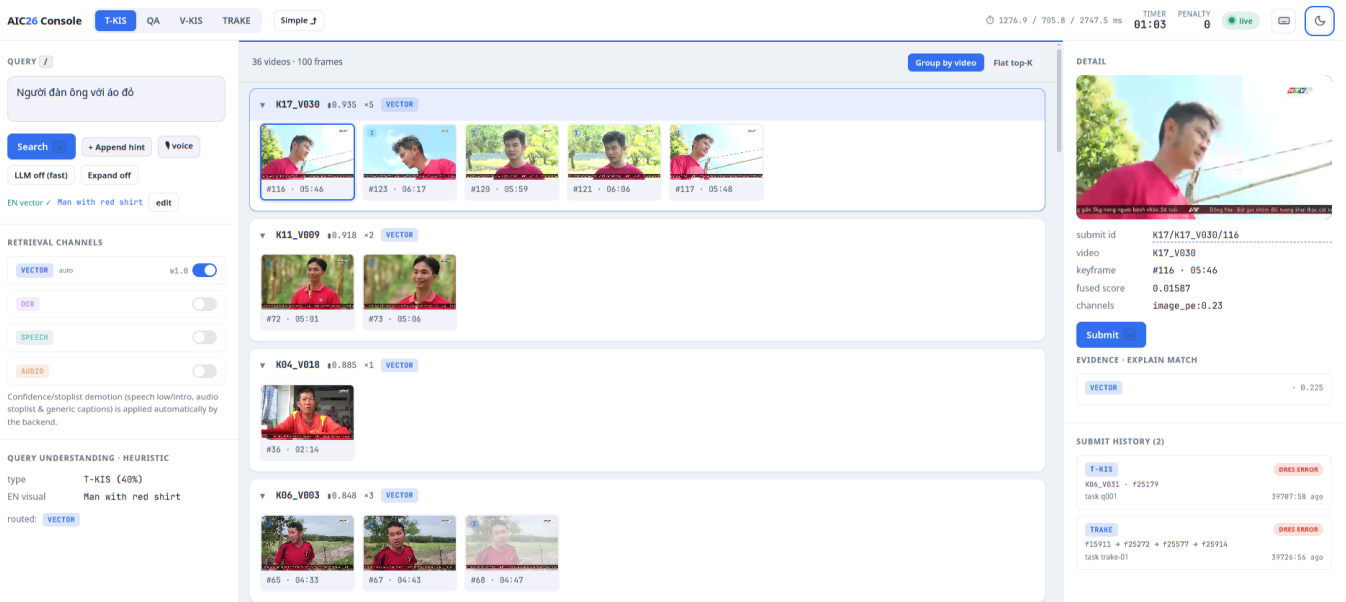


Fig. 7. Operator console for hybrid video retrieval. The interface combines query routing, grouped keyframe results, inline video playback, timeline inspection, evidence snippets, and submission controls in a single screen.

fields. Speech evidence includes transcript, time range, confidence bucket, and segment role. Audio evidence includes top label, caption quality, time range, or GLAP-vector match status. The detail panel exposes the canonical id, score, timing, keyframe thumbnail, and copyable identifier.

C. Voice Input and Keyboard Workflow

The query panel supports browser Web Speech API when available and a backend Whisper endpoint otherwise. Transcribed Vietnamese text is translated for visual search when needed. Keyboard shortcuts minimize mouse travel: / focuses

query, Enter searches or opens the guard, arrow keys move between videos and frames, v opens or hides the video for the selected frame, T toggles timeline, Ctrl+M starts voice input, and Ctrl+/ opens the shortcut modal.

D. Submission and Duplicate Guard

The submit guard is the final safety layer. For KIS/V-KIS/QA, the deduplication key is `video_id:frame_idx`. For TRAKE, it is `video_id|f1,f2,...`. The guard blocks missing frame indices, warns about duplicates, validates increasing event order, displays evidence, and optionally

creates a local result payload. The current system records this payload in local history for review and testing; it does not assume an external evaluation protocol.

IX. IMPLEMENTATION DETAILS

A. Backend

The backend is implemented in Python with FastAPI [22]. Configuration is loaded from environment variables or local secret files for development. Core modules are kept separate from Pydantic/FastAPI request models so ranking logic can be unit tested independently. Adapter modules wrap Elastic, Milvus, PE text encoding, and GLAP text encoding. Services implement search, TRAKE, timeline, and submission history. Mock mode returns deterministic fixtures and is used for frontend development and automated tests without live services.

Important backend endpoints include `/api/health`, `/api/query/parse`, `/api/search`, `/api/search/simple`, `/api/search/trake`, `/api/videos/{video_id}/timeline`, `/api/videos/{video_id}/snap`, `/api/submit`, `/api/submit/history`, `/api/translate`, and `/api/transcribe`.

B. Frontend

The frontend is implemented with React, TypeScript, and Vite. It stores no secrets and talks only to the backend. The main console component owns task type, query, parser output, selected video/frame, timeline cache, video state, TRAKE slots, duplicate guard, and submission history. The video viewer seeks to the selected keyframe timestamp on metadata load and when the selected frame changes. During TRAKE frame picking, pausing the video captures the raw media time and thumbnail; the frame index is computed as $\text{round}(t \cdot \text{fps})$, without snapping back to the nearest extracted keyframe.

C. Storage and Runtime

Elastic Cloud stores sparse metadata and supports BM25/fuzzy queries. Milvus stores dense vectors with cosine search. Cloudflare R2 provides public media URLs. External encoder servers run PE and GLAP text encoders, typically on Kaggle or a tunnelled inference host. The system exposes health warnings when a service is unreachable and marks unavailable capabilities in the UI.

D. Operational Readiness Checks

Before a live session, the deployment verifies that Elastic, Milvus, media, and encoder endpoints are reachable; indexed document/vector counts match the expected corpus state; and a sample result carries valid `submit_keyframe_id`, `video_id`, `keyframe_n`, `pts_time`, `frame_idx`, and media URLs. End-to-end checks also confirm that `v` opens the selected video at the correct timestamp, the submission guard blocks duplicates while allowing deliberate overrides, and a TRAKE query returns an ordered sequence or an explicit partial-result warning.

TABLE II
INDEXED CORPUS STATISTICS.

Artifact	Count	Backend
Videos with temporal maps	1,478	Elastic
Keyframe map records	382,299	Elastic
OCR keyframe records	382,299	Elastic
Speech segments	48,819	Elastic
Audio windows	466,996	Elastic
Audio caption records	35,222	Elastic
PE visual vectors	382,299	Milvus
GLAP audio vectors	466,996	Milvus

X. EXPERIMENTS AND EVALUATION

A. Indexing Statistics

Table II reports the indexed corpus state. These counts were derived from the repository staging summary and handoff verification logs.

B. Retrieval Performance on TKIS Queries

We evaluate the proposed hybrid search approach on the complete TKIS portion of `DanhSachTruy-VanAIC_Chungket.xlsx`. It contains 14 final-round queries: `tkis-query-01-10` and `tkis-query-12-15`; identifier 11 is absent from the supplied sheet. Thus, the set was selected only by task type, rather than by retrieval difficulty or observed performance. To reflect the sequentially revealed challenge setting, we run one mode with the full query and another with its first two sentences. The baseline is standalone CLIP-ViT-H-14 retrieval over the same extracted keyframe corpus.

1) *Evaluation Protocol*: Each query has one manually annotated target `video_id`. Ground truth is therefore video-level, not an exact keyframe or temporal interval. Both methods first retrieve frame candidates, but Recall@K (R@K) is computed on the ranked video groups after grouping. A query is successful at K when its target `video_id` occurs among the first K groups. Any extracted keyframe from that video counts as a hit; retrieving a nearby or different frame in the correct video is not penalized. Consequently, these numbers measure target-video retrieval and must not be interpreted as exact-frame localization accuracy. For both methods, up to 100 frame candidates are retrieved per active channel before grouping, and the list is not truncated by the UI’s display limit before metric computation. The standalone baseline is grouped with the same video aggregation function so that K has the same unit for both methods.

The workbook’s supplied English translation is used as the visual-text query, while the original Vietnamese text is used for deterministic channel routing and for OCR and speech search; no online machine translation is used in the reported TKIS evaluation. Query type is fixed to TKIS. The heuristic parser is used for all queries; LLM parsing and visual query expansion are disabled. PE visual retrieval is always active with weight 1.0. OCR, speech, and audio are activated only when their deterministic cue rules fire, with weights 0.8, 0.8,

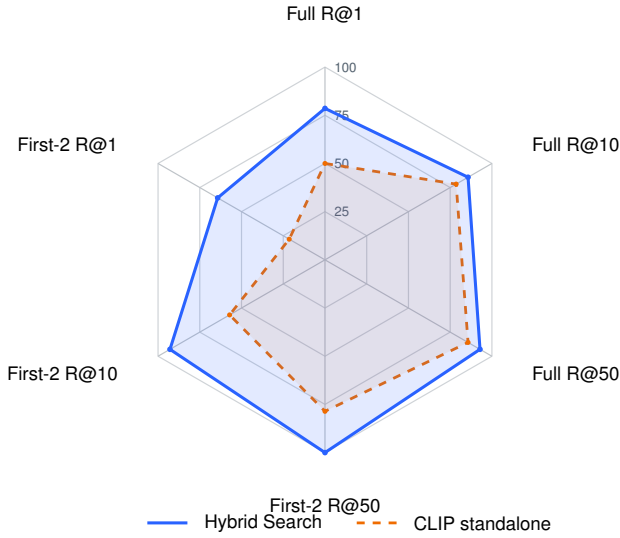


Fig. 8. Retrieval profiles on the 14 TKIS queries at selected cutoffs. Each spoke reports video-level Recall@K (%); First-2 denotes the first-two-sentences condition.

and 0.7, respectively; they are not forced on for every query. Each active channel returns at most 100 frame hits. Frame lists are fused by weighted RRF with $k = 60$, then grouped using best-frame, top-three-mean, and local temporal-support weights of 0.75, 0.15, and 0.10. Relevance feedback, manual channel overrides, and all operator interaction are disabled. The first-two-sentences condition applies the same truncation to the Vietnamese query and its supplied English translation. This fixed configuration is used for every row in Table III; with 14 queries, one query corresponds to 7.14 percentage points.

Table III reports the exact values, while Fig. 8 summarizes selected cutoffs as retrieval profiles. The results show a consistent advantage for the hybrid system over the standalone CLIP baseline. Under full-query evaluation, hybrid search improves early retrieval most strongly: R@1 increases from 50.00% to 78.57%, meaning that the correct target video becomes the first group for four more queries out of 14. The gain remains visible at larger cutoffs, reaching 92.86% at R@50 and full recall at R@100. This pattern suggests that combining visual-text embeddings with selectively routed OCR, speech, and audio evidence, followed by rank fusion and video grouping, reduces early-ranking mistakes while keeping broad recall.

The incomplete-query setting makes the robustness difference clearer. When only the first two sentences are used, standalone CLIP drops sharply to 21.43% R@1, while hybrid search sustains 64.29% R@1 and reaches 100.00% by R@50. This is important for interactive search because operators often issue an initial query before all query details are fully processed. Auxiliary modalities help recover from missing context: OCR can match visible names and numbers, metadata can anchor likely videos or time ranges, and speech or audio evidence can contribute when the query contains those cues. The result should still be interpreted cautiously because the

evaluation set is small and modality-level ablations are not yet reported. In particular, this table does not isolate or attribute gains to the optional caption and object enrichment extensions. The observed trend supports multimodal fusion over a standalone visual-text baseline.

C. TRAKE Retrieval Evaluation

The supplied workbook contains four TRAKE queries, and we evaluate the complete set rather than a selected subset. Because it does not include machine-readable target frames, each first-ranked result is manually inspected. A query is counted as correct at R@1 only when the top-ranked video contains one visually matching frame for every described event and the selected timestamps are strictly increasing. Table IV reports 4/4 correct first-ranked sequences and full coverage of all 14 events. This is an operator-verified sequence-level result on a small set, not an official test-server score.

Controlled greedy-failure trace. Table V evaluates the specific failure mode targeted by the sequence DP using a controlled three-event candidate trace. The greedy baseline takes the highest-scoring candidate for each event that remains later than its previous pick. It chooses E_1 at 50 s and E_2 at 70 s, leaving no valid candidate for E_3 at 60 s. The proposed DP selects the lower-scoring early candidates at 10 s and 30 s, then reaches E_3 at 60 s. This fixture is executed by the same backend sequence assembler in the unit tests.

The controlled result demonstrates that temporal coverage can require rejecting a locally stronger earlier event. Broader TRAKE conclusions still require a held-out set with machine-readable video and event-time annotations, comparison against retrieval baselines, and repeated live runs for end-to-end latency.

D. Backend Runtime Diagnostics

Table VI reports warm backend latency on the live deployment. Each retrieval operation uses one warm-up followed by 25 measured requests with 100 candidates per active channel. Table VI reports median, nearest-rank p95, and min-max wall-clock latency from the backend host. The local heuristic router remains below 0.1 ms over 100 calls and is omitted from the table. The full-hybrid row activates PE, OCR, speech, and Elastic+GLAP audio concurrently and includes timing enrichment, weighted RRF, and video grouping. The accompanying `benchmark_runtime.py` runner uses fixed channel queries and excludes parser and translation overhead. External service load and network conditions are therefore part of all measurements.

These values characterize service responsiveness, not human search efficiency. Browser video open/seek latency, time-to-correct-result, duplicate attempts, and wrong-frame selections were not instrumented in this evaluation.

E. Module Validation

The project validates modules at several levels. Upload scripts include unit tests for keyframe mapping, Elastic document construction, OCR normalization, Milvus entity schemas,

TABLE III
VIDEO-LEVEL RETRIEVAL PERFORMANCE AFTER GROUPING ON ALL 14 TKIS QUERIES.

Method	Full Query					First Two Sentences Query				
	R@1	R@5	R@10	R@50	R@100	R@1	R@5	R@10	R@50	R@100
Hybrid Search (ours)	78.57%	85.71%	85.71%	92.86%	100.00%	64.29%	85.71%	92.86%	100.00%	100.00%
CLIP-ViT-H-14 (standalone)	50.00%	71.43%	78.57%	85.71%	85.71%	21.43%	57.14%	57.14%	78.57%	85.71%

TABLE IV

OPERATOR-VERIFIED TRAKE PERFORMANCE ON ALL SUPPLIED QUERIES. EVENT COVERAGE IS MEASURED WITHIN THE FIRST-RANKED SEQUENCE.

Query	Ordered event scenario	Events	R@1 / coverage
trake-01	Character-costume appearances	4	Correct; 4/4
trake-02	Cycling finish order	3	Correct; 3/3
trake-03	Philippines-Brunei match events	3	Correct; 3/3
trake-04	Cooking ingredient order	4	Correct; 4/4
Overall	All supplied TRAKE queries	14	4/4; 14/14

TABLE V

CONTROLLED SYNTHETIC TRAKE TRACE. SCORES IN PARENTHESES ARE RETRIEVED CANDIDATE RELEVANCE. THE EXAMPLE VALIDATES GREEDY FAILURE, NOT CORPUS-LEVEL TRAKE ACCURACY.

Method	E_1	E_2	E_3	Valid order / coverage
Greedy	50 s (0.95)	70 s (0.93)	—	No; 2/3
Proposed DP	10 s (0.80)	30 s (0.83)	60 s (0.90)	Yes; 3/3

TABLE VI

WARM BACKEND LATENCY ON THE LIVE DEPLOYMENT ($n = 25$ MEASURED REQUESTS PER RETRIEVAL OPERATION AFTER ONE WARM-UP).

Operation	Median	p95	Min-max
PE visual retrieval	1.38 s	1.64 s	1.33–1.66 s
OCR retrieval	1.65 s	2.17 s	1.60–2.69 s
Speech retrieval	1.64 s	2.64 s	1.60–3.02 s
Audio retrieval	2.51 s	2.97 s	2.42–3.12 s
Full four-channel hybrid	2.55 s	3.36 s	2.48–4.38 s

and retry behavior. Backend tests cover identity parsing, media URL construction, query parsing, scoring demotions, RRF fusion, group-by-video ranking, TRAKE dynamic programming, adapter mock behavior, timeline construction, duplicate submission blocking, translation fallback, and API responses. Frontend component tests cover search flow, grouped and flat views, keyboard selection, video toggle with v, TRAKE quick submit, duplicate guard, and simple vector search.

Extraction-specific validation was performed without exhaustive manual labels. For keyframe extraction, validation checks map-file monotonicity, valid `frame_idx` and `pts_time` ranges, stability-score distributions, duplicate rates after temporally decayed coverage, and successful joins from extracted frames to media URLs. For OCR, the pipeline verified 382,299 unique records with no duplicate ids and tracked the reduction from cleanup. For speech, alignment confidence distributions were inspected and low-score segments were preserved for indexing time demotion. For audio, window

TABLE VII

VERIFIED RETRIEVAL CASES FROM A POST-HOC LIVE RERUN. V/H DENOTE PE-ONLY AND FULL-HYBRID VIDEO-GROUP RANKS; THE SPEECH AND AUDIO DIAGNOSTICS ARE OUTSIDE THE 14-QUERY QUANTITATIVE SET.

Case	Target and observed rank	Query cue and decisive evidence
OCR (TKIS-03)	K13_V001/016; V >100 → H 68	The query asks for stage 01 of the television cycling race in Tuyên Quang. OCR reads “ĐOÀN ĐUA CÚP TRUYỀN HÌNH ĐẾN TUYÊN QUANG,” bringing an otherwise missed target into the top 100.
Speech (TKIS-09)	K14_V025/202; V 11 → H 11	The query describes villagers in Indonesia sleeping on sand. ASR at 595.2–615.4 s mentions a rural village on Madura island, Indonesia, and the long-standing practice of sleeping on sand.
Speech diagnostic	K06_V005/196; V 28 → H 12	“Lời thuyết minh nói lễ hội bắt nguồn từ một lễ hội tôn giáo vào thế kỷ thứ mười chín để tôn vinh Đức Mẹ Maria.” ASR at 763.878–788.153 s states this origin explicitly, improving the target by 16 video-group positions.
Audio diagnostic	L24_V023/047; V 19 → H 15	“Tiếng chuông kim loại vang liên tục, đều nhịp.” The 162.5–167.5 s window has label <code>Bell1</code> and a caption describing continuous rhythmic metallic ringing resembling a bell or gong.

integrity was checked across 1,478 videos, and tag/caption quality analysis motivated stoplists and caption-quality fields. For structured captions, validation focuses on JSON parse rate, hard-field vocabulary coercion, near-duplicate inheritance, and spot checks of text-independent V-KIS fields. For object detection, weak validation compares overlapping classes with provided object metadata where available, measures cross-detector agreement, and renders bbox overlays for genre-vocabulary and threshold tuning.

F. Retrieval Case Studies

The aggregate evaluation retained Recall@K values but not its original per-query rank log. After restoring the PE encoder, we therefore reran four post-hoc cases with the fixed protocol above: 100 candidates per active channel, weighted RRF, video grouping, and no feedback, expansion, or manual override. Targets were verified independently from indexed OCR, speech, or audio evidence. The additional speech diagnostic was constructed from a distinctive fact in an indexed ASR segment to isolate the speech channel; it is not one of the 14 official TKIS queries. Table VII reports actual PE-only and full-hybrid video-group ranks from this rerun. These cases explain individual behavior but are not an additional aggregate benchmark.

The OCR case is a genuine rescue at the R@100 cutoff: PE alone does not return the target, whereas the location-specific

TABLE VIII
RECOMMENDED FUTURE ABLATIONS.

Variant	Removed component	Hypothesis (not measured)
Visual only	Dense PE without OCR/speech/audio	Misses exact text, spoken phrases, and sound cues
No OCR	Remove OCR channel from full system	Lower recall for lower-thirds, signage, names, numbers, clocks
No speech	Remove ASR channel	Lower recall for spoken descriptions and QA evidence
No audio vector	Keep Elastic audio tags/captions, remove GLAP	Lower recall for semantic sound queries and noisy tags
No grouping	Show flat fused frames only	Repeated videos may increase inspection effort
Old count-heavy grouping	Replace best-frame-dominant score	Videos with many mediocre frames overtake strong candidates
Greedy TRAKE	Replace DP with left-to-right greedy picks	Locally strong early event can block a globally valid sequence
No submit guard	Submit directly from selected result	Duplicate and wrong-frame risk may increase

title makes it reachable through OCR. The official TKIS speech case does not improve the video rank and is reported as such; instead, it changes the representative evidence from a visually plausible frame to a transcript-backed moment, making verification less ambiguous. The separate speech diagnostic improves the target from rank 28 to 12 because its defining fact is spoken rather than visually observable. Since this query was formed from inspected ASR evidence, the result demonstrates channel behavior but must not be treated as an unbiased benchmark gain. The audio case improves the target by four positions, but is likewise labeled as a diagnostic probe because none of the 14 supplied TKIS descriptions defines a sound event as its target. These cases expose both gains and non-gains rather than attributing every successful query to multimodal fusion.

G. Planned Ablation Protocol

Table VIII defines a planned ablation protocol for future held-out evaluation. No quantitative ablation results are claimed in this work. Because official competition ground truth is not part of this repository, these ablations are suitable for a future validation set of KIS/QA/TRAKE tasks.

XI. LIMITATIONS AND FUTURE WORK

No controlled interaction study. Keyboard navigation, grouped video results, timeline inspection, and the submission guard are implemented and covered by functional tests, but this paper does not measure their human-facing effect. In particular, no flat-versus-grouped comparison, guard ablation, or multi-operator study reports time-to-correct-result, duplicate attempts, or wrong-frame selections. We therefore treat these components as system features, not as experimentally established improvements to live search performance.

No official test-server score or retained TKIS per-query log. Table III is an internal video-level evaluation over the supplied final-round TKIS list; it is not an official leaderboard result, and its original run retained aggregate Recall@K rather than per-query ranks. A future paper version should persist the full ranked lists and report MRR, Recall@K, successful

submission rate, median time-to-correct-submit, and wrong-submit rate on a task suite.

No live VLM reranker or visual QA chain-of-thought. The current system does not run a top- k VLM reranker or image-based QA reasoning. The backend has clean extension points for such a reranker, but health flags currently mark these capabilities as unavailable.

Vietnamese analysis remains simple. Elastic text fields currently use standard text mappings and folded fields rather than a custom Vietnamese analyzer. This is robust enough for many queries but may underperform on morphologically complex or diacritic-error cases.

Audio labels are noisy. CED tags include systematic errors on Vietnamese speech. Stoplists and demotions reduce but do not eliminate this noise. GLAP provides a second retrieval path, but GLAP availability depends on the external encoder endpoint.

Video loading performance. Inline video loading depends on remote MP4 layout and network behavior. Remuxing videos with fast-start metadata, adding lower-bitrate preview proxies, and delaying timeline thumbnails until video metadata is ready would improve operator latency.

Caption/object accuracy lacks labeled evaluation. The keyframe extractor is part of the offline corpus construction used by the retrieval system, while captioning and object grounding are enrichment branches over the same extracted frame registry. Their remaining limitation is evaluation and serving depth: without exhaustive caption/object labels, the paper reports schema design, weak validation, and integration contracts rather than mAP or caption-relevance scores.

XII. CONCLUSION

This paper presented AIC26-Retrieval as an adaptive-keyframe and evidence-aware hybrid retrieval system for AIC 2026. The offline method constructs representative frames while preserving a canonical frame-time identity across visual, OCR, speech, and audio indices. Online cue routing, weighted RRF, and video grouping raise full-query video-level R@1 from 50.00% to 78.57% over the standalone baseline and retain a larger advantage when query text is truncated. The TRAKE branch further uses exact dynamic programming for globally ordered event assembly; all four supplied TRAKE queries return an operator-verified correct sequence at R@1, with 14/14 event coverage. Across 25 post-warm-up requests, the four-channel hybrid path records 2.55 s median and 3.36 s p95 backend latency. The interface, timeline, grouping workflow, and submission guard complete the deployed system, but their effects on search time and submission errors require a controlled multi-operator study before making interaction-performance claims.

REFERENCES

- [1] L. Rossetto, K. Schoeffmann, C. Gurrin, J. Lokoc, and W. Bailer, "Results of the 2024 video browser showdown," *arXiv preprint arXiv:2502.15683*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.15683>

- [2] —, “Results of the 2025 video browser showdown,” *arXiv preprint arXiv:2509.12000*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.12000>
- [3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2021, pp. 8748–8763. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [4] Hunyuan Vision Team, P. Lyu, X. Wan, G. Li, S. Peng, W. Wang, L. Wu, H. Shen, Y. Zhou, C. Tang, Q. Yang, Q. Peng, B. Luo, H. Yang, H. Peng, H. Yang, S. Xie, B. Wu, M. Yang, S. Wang, D. Zhu, J. Jiang, Linus, H. Hu, and C. Zhang, “HunyuanOCR technical report,” *arXiv preprint arXiv:2511.19575*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.19575>
- [5] T.-T. Le, L. T. Nguyen, and D. Q. Nguyen, “PhoWhisper: Automatic speech recognition for vietnamese,” *arXiv preprint arXiv:2406.02555*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.02555>
- [6] M. Bain, J. Huh, T. Han, and A. Zisserman, “Whisperx: Time-accurate speech transcription of long-form audio,” *arXiv preprint arXiv:2303.00747*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.00747>
- [7] H. Dinkel, Z. Yan, T. Wang, Y. Wang, X. Sun, Y. Niu, J. Liu, G. Li, J. Zhang, and J. Luan, “GLAP: General contrastive audio-text pretraining across domains and languages,” *arXiv preprint arXiv:2506.11350*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.11350>
- [8] H. Dinkel, Y. Wang, Z. Yan, J. Zhang, and Y. Wang, “CED: Consistent ensemble distillation of audio tagging,” *arXiv preprint arXiv:2308.11957*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.11957>
- [9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759.
- [10] T. Souček and J. Lokoč, “TransNet V2: An effective deep network architecture for fast shot transition detection,” *arXiv preprint arXiv:2008.04838*, 2020. [Online]. Available: <https://arxiv.org/abs/2008.04838>
- [11] Elastic, “Elasticsearch: The distributed search and analytics engine,” <https://www.elastic.co/elasticsearch>, 2026, accessed July 2026.
- [12] Milvus Project, “Milvus: An open-source vector database,” <https://milvus.io/>, 2026, accessed July 2026.
- [13] D. Bolya, P.-Y. Huang, P. Sun, J. H. Cho, A. Madotto, C. Wei, T. Ma, J. Zhi, J. Rajasegaran, H. Rasheed, J. Wang, M. Monteiro, H. Xu, S. Dong, N. Ravi, D. Li, P. Dollár, and C. Feichtenhofer, “Perception Encoder: The best visual embeddings are not at the output of the network,” *arXiv preprint arXiv:2504.13181*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.13181>
- [14] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 11 975–11 986.
- [15] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 1996, pp. 226–231.
- [16] Z. Liu, L. Zhu, B. Shi, Z. Zhang, Y. Lou, S. Yang, H. Xi, S. Cao, Y. Gu, D. Li, X. Li, Y. Fang, Y. Chen, C.-Y. Hsieh, D.-A. Huang, A.-C. Cheng, V. Nath, J. Hu, S. Liu, R. Krishna, D. Xu, X. Wang, P. Molchanov, J. Kautz, H. Yin, S. Han, and Y. Lu, “NVILA: Efficient frontier visual language models,” *arXiv preprint arXiv:2412.04468*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.04468>
- [17] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “YOLOE: Real-time seeing anything,” *arXiv preprint arXiv:2503.07465*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.07465>
- [18] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024.
- [19] R. Solovyev, W. Wang, and T. Gabruseva, “Weighted boxes fusion: Ensembling boxes from different object detection models,” *Image and Vision Computing*, vol. 107, p. 104117, 2021.
- [20] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jégou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024.
- [21] H. Dinkel, G. Li, J. Liu, J. Luan, Y. Niu, X. Sun, T. Wang, Q. Xiao, J. Zhang, and J. Zhou, “MiDashengLM: Efficient audio understanding with general audio captions,” *arXiv preprint arXiv:2508.03983*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.03983>
- [22] FastAPI, “Fastapi framework documentation,” <https://fastapi.tiangolo.com/>, 2026, accessed July 2026.